

CRT Phase 2 - Post Test

Name: Aditya Vishal Kayasth

Q1. Predict the output. What will print to the console when this Spring Boot application starts? Write the output in order and give a one-line reason for each. [4M]

```
@Component
public class MyBean {
    public MyBean() {
        System.out.println("Constructor called");
    }

    @PostConstruct
    public void init() {
        System.out.println("Init called");
    }
}
```

Output:

Constructor called
Init called

Reason: Spring IoC container first creates the bean instance using the constructor, and then invokes methods annotated with `@PostConstruct` after dependency injection is fully complete.

Q2. The endpoint below compiles and runs but always returns an empty response body. Identify the mistake and write the corrected line only. [3M]

```
@Controller
@RequestMapping("/api/users")
public class UserController {
    @Autowired UserService service;

    @GetMapping("/{id}")
    public ResponseEntity<User> getUser(@PathVariable Long id) {
        User u = service.findById(id);
        return new ResponseEntity<>(HttpStatus.OK);
    }
}
```

Two problems:

1. Use `@RestController` instead of `@Controller`
 2. The return statement should be `new ResponseEntity.ok(u);`
-

Q4. What annotation should be added to the transfer() method below, and why? Write the annotation and explain in 2 lines what happens if it is missing. [3M]

```
@Service
public class BankService {
    @Autowired AccountRepository repo;

    public void transfer(Long fromId, Long toId, double amount) {
        Account from = repo.findById(fromId).orElseThrow();
        Account to = repo.findById(toId).orElseThrow();
        from.setBalance(from.getBalance() - amount);
        to.setBalance(to.getBalance() + amount);
        repo.save(from);
        repo.save(to);
    }
}
```

Missing Annotation: @Transactional above transfer method.

Reason: If missing, any exception occurring during the transfer (e.g., deducting from one account but failing to add to the other) will not trigger a rollback, leading to inconsistent database states.

Q6. Look at the JWT filter below. (a) Explain in 2 lines what this filter does. (b) There is one bug — the token extracted from the header will always be invalid. Find it and write the corrected line. [5M]

```
String header = req.getHeader("Authorization");
if (header == null || !header.startsWith("Bearer ")) {
    chain.doFilter(req, res);
    return;
}
String token = header.substring(6); // extract token
String username = jwtUtil.extractUsername(token);
if (username != null) {
    UsernamePasswordAuthenticationToken auth =
        new UsernamePasswordAuthenticationToken(username, null, null);
    SecurityContextHolder.getContext().setAuthentication(auth);
}
chain.doFilter(req, res);
```

- a) This filter checks incoming HTTP requests for an "Authorization" header containing a JWT. If a valid token is present, it extracts the username and registers the user as authenticated within the Spring Security context for that specific request.
- b) **The Bug:** The prefix being checked is "Bearer " (which includes a space). This prefix is exactly 7 characters long (indices 0 through 6). By using header.substring(6), the extracted token will incorrectly start with a leading space (e.g., " eyJhbGci..."), which makes the JWT invalid.

Q7. Write a Spring Data JPA repository method using @Query to find all Product entities where price is less than a given value and category matches a given string. Use named parameters.

```
@Query("SELECT p FROM Product p WHERE p.price < :price AND p.category =  
:category")  
List<Product> findByPriceAndCategory(@Param("price") double price, @Param("category")  
String category);
```

Q8. Explain the N+1 query problem in JPA in your own words with a simple example. [4M]
What is the most common cause? Write one line of code that fixes it in a @Query method
using JOIN FETCH.

Explanation: The N+1 problem happens when your ORM executes 1 query to fetch a list of
parent entities, and then executes N additional queries to fetch the child entities for each
parent one by one.

Common Cause: Iterating over a collection of related entities that are mapped with
FetchType.LAZY.

Q9. Write a SQL query to find the second highest salary from a table employees(id, name,
salary). Use a subquery approach. Then write the same logic as a JPQL @Query method
in a JPA repository. [4M]

SQL

```
SELECT MAX(salary) FROM employees  
WHERE salary < (SELECT MAX(salary) FROM employees);
```

JPQL

```
@Query("SELECT MAX(e.salary) FROM Employee e WHERE e.salary < (SELECT  
MAX(e2.salary) FROM Employee e2)")  
Double findSecondHighestSalary();
```

Q10. Given a Student entity with fields name, email, and city, write FOUR Spring Data
JPA derived query method names (no @Query) for the following: (a) find by city, (b) find by
name and city, (c) find by email containing a keyword, (d) count students in a city. Write
the method signatures only. [5M]

```
List<Student> findByCity(String city);  
List<Student> findByNameAndCity(String name, String city);  
List<Student> findByEmailContaining(String keyword);  
long countByCity(String city);
```

Q11. A developer runs the following code to load 10 orders and print their items. The

console shows 11 SQL queries instead of 1. What is this problem called? Write the line of code that causes it and the one-line fix.

```
List<Order> orders = orderRepo.findAll();
for (Order o : orders) {
    System.out.println(o.getItems().size());
}
```

This happens because 1 query is executed to load the 10 orders, and then 10 separate queries are executed to load the items for each order.

This happens because 1 query is executed to load the 10 orders, and then 10 separate queries are executed to load the items for each order.

```
System.out.println(o.getItems().size());
```

To fix this, you must fetch the orders and their associated items in a single query using JOIN FETCH in your repository interface.

```
@Query("SELECT o FROM Order o JOIN FETCH o.items")
List<Order> findAllWithItems();
```

Q12. Write the application.yml configuration and the main class annotation needed to register a Spring Boot service named product-service with a Eureka server at <http://localhost:8761/eureka>.

[4M]

YAML:

```
spring:
  application:
    name: product-service
```

```
eureka:
  client:
    serviceUrl:
      defaultZone: http://localhost:8761/eureka
```

The main class Should have `@EnableDiscoveryClient` annotation

Q13. What is a Circuit Breaker in microservices? Why is it needed? Write a Spring Boot method that calls an external payment-service using RestTemplate and applies a Resilience4j `@CircuitBreaker` with a fallback method that returns a default PaymentResponse object.

[5M]

What it is: A Circuit Breaker is a design pattern used in microservices that acts as a proxy for operations that might fail. It monitors the number of recent failures; if they cross a certain threshold, the circuit breaker "trips" (opens). While open, it immediately rejects further calls without even trying to execute them, instead returning a predefined fallback response. After a timeout period, it allows a limited number of test requests through (half-open state) to see if the underlying issue is resolved.

Why it is needed: In a distributed architecture, services rely on each other. If one service (e.g., payment-service) goes down or becomes extremely slow, requests from calling services will pile up, consuming threads and memory. A circuit breaker prevents these cascading failures across the system, ensuring the application remains responsive and can gracefully degrade its functionality instead of completely crashing.

Q14. What is an API Gateway and what are 2 benefits of using one? Write a Spring Cloud Gateway application.yml that routes requests starting with /api/orders/** to order-service and /api/users/** to user-service. Use lb:// for load-balanced URIs. [6M]

An API Gateway acts as a single, unified entry point for external clients (like web or mobile apps) to access a backend system of microservices. Instead of clients calling dozens of different microservices directly, they send all requests to the Gateway, which then routes them to the appropriate destination.

Client Abstraction: It hides the complexity of the backend architecture. Clients don't need to know the IP addresses or hostnames of individual microservices; they only need to know the Gateway's address.

Centralized Cross-Cutting Concerns: It provides a single place to handle global logic such as authentication/authorization (e.g., validating JWTs), rate limiting, and CORS, preventing you from having to duplicate this logic in every single microservice.

YAML:

```
spring:
  cloud:
    gateway:
      routes:
        - id: order-service-route
          uri: lb://order-service
          predicates:
            - Path=/api/orders/**

        - id: user-service-route
          uri: lb://user-service
          predicates:
            - Path=/api/users/**
```

Q15. What is Service Discovery in microservices and why is it needed instead of hardcoding IP addresses? Name the tool you used in class. Write a RestTemplate bean configuration that enables it to resolve service names automatically.

[4M]

Service Discovery acts as a dynamic phonebook for your microservices (typically using **Netflix Eureka**), allowing them to locate each other by name rather than relying on fragile, hardcoded IP addresses that change whenever a service restarts. Think of it like saving a contact in your phone—you dial the name, not the exact digits, so it still works even if they change their number!

JAVA:

```
import org.springframework.cloud.client.loadbalancer.LoadBalanced;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.web.client.RestTemplate;
```

```
@Configuration
```

```
public class AppConfig {
```

```
    @Bean
```

```
    @LoadBalanced
```

```
    public RestTemplate restTemplate() {
```

```
        return new RestTemplate();
```

```
    }
```

```
}
```

Q16. You have a monolithic Spring Boot app with modules: UserModule, OrderModule, ProductModule. Your team wants to split it into microservices. (a) List 3 benefits of this split. (b) List 2 challenges. (c) Briefly explain how Eureka and an API Gateway work together so a React frontend can talk to all three services through a single URL.

[6M]

Splitting a monolith into microservices allows individual pieces of your application to scale and update independently, though it requires more advanced infrastructure to keep everything connected. Here is the breakdown of the benefits, challenges, and how the routing architecture works.

(a) 3 Benefits of Splitting into Microservices:

1. **Independent Deployment:** You can update and deploy the UserModule without needing to redeploy the entire application.
2. **Independent Scaling:** If the OrderModule gets a massive spike in traffic, you can allocate more resources specifically to it without paying to scale the other modules.

3. Fault Isolation: If a bug causes the ProductModule to crash, the user and order systems can remain online and functional.

(b) 2 Challenges:

1. Operational Complexity: Managing, deploying, monitoring, and debugging multiple interconnected services is much harder than running a single monolith.
2. Distributed Data: Handling database transactions that span multiple services (e.g., placing an order and updating product inventory simultaneously) becomes very difficult.

(c) How Eureka and API Gateway Work Together:

The React frontend only needs to know the single URL of the API Gateway, sending all its requests there. When the Gateway receives a request (e.g., /api/orders), it asks Eureka (the service registry) for the current, live IP address of the order-service, and then securely forwards the request to that specific destination.

Q18. What is the difference between @Mock (Mockito) and @MockBean (Spring Boot Test)? When would you use each? Write one example of a test class declaration that uses @MockBean correctly with @SpringBootTest. [4M]

@Mock creates a lightweight, isolated mock object for fast unit tests without starting the Spring framework. @MockBean is used specifically in Spring integration tests to load the entire application context but replace a real internal component (like a service) with a mock version.

JAVA:

```
import org.springframework.boot.test.context.SpringBootTest;
import org.springframework.boot.test.mock.mockito.MockBean;
```

```
@SpringBootTest
public class OrderControllerIntegrationTest {

    // This replaces the real OrderService bean in the loaded Spring Context with a mock
    @MockBean
    private OrderService orderService;

}
```

Q19. Write a Dockerfile for a Spring Boot application. The JAR file is named app.jar and is inside the target/ folder. Use eclipse-temurin:21-jre as the base image. The app runs on port 8080. Then write the docker build and docker run commands to build the image as myapp:latest and run it. [5M]

A Dockerfile provides the blueprint to package your compiled Spring Boot application into a portable, lightweight container. The terminal commands then use this blueprint to build the image and spin up a running instance mapped to your local port.

DOCKER:

```
# Dockerfile
FROM eclipse-temurin:21-jre
COPY target/app.jar app.jar
EXPOSE 8080
ENTRYPOINT ["java", "-jar", "app.jar"]
```

BASH:

```
# Terminal Commands
docker build -t myapp:latest .
docker run -p 8080:8080 myapp:latest
```

Q20. Write a docker-compose.yml with two services: (a) db — using the postgres:15 image with environment variables for DB name, user, and password, and (b) app — using the myapp:latest image, mapping port 8080, connecting to the db service, and starting only after db starts using depends_on. [6M]

This docker-compose.yml orchestrates both containers, using environment variables to set up the database credentials and allowing the Spring Boot app to connect using the service name db as the host. The depends_on instruction guarantees the database initializes before the application starts up.

YAML:

```
version: '3.8'

services:
  db:
    image: postgres:15
    environment:
      POSTGRES_DB: mydb
      POSTGRES_USER: dbuser
      POSTGRES_PASSWORD: dbpassword

  app:
    image: myapp:latest
```



```
ports:
  - "8080:8080"
depends_on:
  - db
environment:
  # This demonstrates how the app connects to the 'db' service
  SPRING_DATASOURCE_URL: jdbc:postgresql://db:5432/mydb
  SPRING_DATASOURCE_USERNAME: dbuser
  SPRING_DATASOURCE_PASSWORD: dbpassword
```

Q21. Write a JavaScript async function `loadUsers()` that fetches data from `GET /api/users` using the `fetch` API, checks if the response is successful (`response.ok`), and either renders a list of user names into a `<ul id="user-list">` element or shows an error message in a `<p id="error">` element. [5M]

This asynchronous function uses a `try...catch` block to make the API call and gracefully handle any network failures. It checks `response.ok` to ensure a successful HTTP status code before mapping the returned data into HTML list items.

```
async function loadUsers() {
  try {
    const response = await fetch('/api/users');
    if (!response.ok) {
      throw new Error('Network response was not ok');
    }
    const users = await response.json();
    const list = document.getElementById('user-list');
    list.innerHTML = users.map(u => `<li>${u.name}</li>`).join("");
  } catch (error) {
    document.getElementById('error').textContent = 'Failed to load users.';
  }
}
```

Q22. The React component below has a bug that causes it to fetch data in an infinite loop. [5M]
(a) Identify exactly what is wrong. (b) Fix it. (c) Add a simple loading state that shows 'Loading...' while data is being fetched.

```
function UserList() {
  const [users, setUsers] = useState([]);

  useEffect(() => {
    fetch('/api/users')
      .then(res => res.json())
      .then(data => setUsers(data));
  }); // <-- notice anything?

  return <ul>{users.map(u => <li key={u.id}>{u.name}</li>)}</ul>;
}
```

The infinite loop occurs because the `useEffect` hook is missing a dependency array, causing it to re-run every time the component renders (which happens continuously because the fetch updates the state via `setUsers`). Adding an empty array `[]` ensures the fetch only happens once when the component mounts, and a new boolean state can easily handle the "Loading..." message.

```
function UserList() {
  const [users, setUsers] = useState([]);
  const [loading, setLoading] = useState(true); // (c) Add loading state

  useEffect(() => {
    fetch('/api/users')
      .then(res => res.json())
      .then(data => {
        setUsers(data);
        setLoading(false); // Stop loading once data is fetched
      })
      .catch(() => setLoading(false));
  }, []); // (b) Fix: Added the empty dependency array here

  // (c) Show simple loading state
  if (loading) return <p>Loading...</p>;

  return <ul>{users.map(u => <li key={u.id}>{u.name}</li>)}</ul>;
}
```
